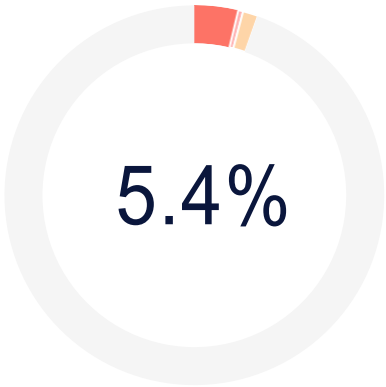
Total Pages 12 Total Words 2950

Plagiarism Report



Results (10)

*Results may not appear because the feature has been disabled.

Private Cloud Hub	Shared Data Hub	Filtered / Excluded
0	0	0
Internet Sources	**AI Source Match**	**Current Batch**
5	0	5

Plagiarism types	Text coverage	Words
Identical	3.7%	109
Minor Changes	0.4%	12
Paraphrased	1.3%	38
Excluded		
Omitted Words		0

About Plagiarism Detection

Our AI-powered plagiarism scans offer three layers of text similarity detection: Identical, Minor Changes, and Paraphrased. Based on your scan settings we also provide insight on how much of the text you are not scanning for plagiarism (Omitted words).

Identical

One to one exact word matches. Learn more

Minor Changes

Words that hold nearly the same meaning but have a change to their form (e.g. “large” becomes “largely”). Learn more

Paraphrased

Different words that hold the same meaning that replace the original content (e.g. "large" becomes "big") Learn more

Omitted Words

The portion of text that is not being scanned for plagiarism based on the scan settings. (e.g. the 'Ignore quotations' setting is enabled and the document is 20% quotations making the omitted words percentage 20%) Learn more

Copyleaks Shared Data Hub

Our Shared Data Hub is a collection of millions of user-submitted documents that you can utilize as a scan resource and choose whether or not you would like to submit the file you are scanning into the Shared Data Hub. Learn more

Filtered and Excluded Results

The report will generate a complete list of results. There is always the option to exclude specific results that are not relevant. Note, by unchecking certain results, the similarity percentage may change. Learn more

Current Batch Results

These are the results displayed from the collection, or batch, of files uploaded for a scan at the same time. Learn more

Plagiarism Detection Results: (10)

BCA-EG.pdf — 1.5%

https://uou.ac.in/sites/default/files/slm/bca-eg_pdf

BCA-EG Artificial Intelligence School of Computer Science & IT Uttarakhand Open University, Haldwani Uttarakhand Open University Behind...

60792.pdf — 1.2%

https://mediatum.ub.tum.de/doc/1083789/60792.pdf

TECHNISCHE UNIVERSITÄT MÜNCHEN Lehrstuhl für Bildverstehen und wissensbasierte Systeme Institut für Informatik Representation and parall...

papers.pdf — 0.9%

https://webdocs.cs.ualberta.ca/~holte/heuristics/papers.pdf

This document gives full references to the papers cited in the slides for Robert Holte's AAAI 2005 tutorial "Where Do Heuristics Come Fro...

Compi.pdf — 0.8%

https://webdocs.cs.ualberta.ca/~jonathan/publications/ai_publications/compi.pdf

kulchits

Computational Intelligence, Volume 14, Number 3, 1998 PATTERN DATABASES Joseph C. Culberson and Jonathan Schaeffer Department of Computi...

Your File — 0.7%

No introduction available.

Your File — 0.6%

Un-named

Courville, Deep Learning, MIT Press, 2016.

Your File — 0.6%

No introduction available.

(PDF) Towards Learning Rubik's Cube with N-tuple-based Reinforcement Learning — 0.6%

https://www.researchgate.net/publication/371314405_towards_learning_rubik's_cube_with_n-tuple-based_reinf...

Home Statistical Learning Biosignal Processing Biosignals Medicine Physiology Reinforcement Learning PreprintPDF AvailableTowards L...

Your File 0.5%

Garima

No introduction available.

Your File 0.4%

No introduction available.

High Performance Rubik's Cube Solver using Optimized Search Algorithms

Ansh Oberoi
Department of Computer Science and Engineering
Chitkara University
Punjab, India
ansh138.be22@chitkara.edu.in

Ravneet Kaur
Department of Computer Science and Engineering
Chitkara University
Punjab, India
ravneet720.be22@chitkara.edu.in

Dr. Shikha Tuteja
Department of Computer Science and Engineering
Chitkara University
Punjab, India
shikha.1290@chitkara.edu.in

Abstract - As a classic example of a complex combinatorial puzzle, the Rubik's Cube presents significant hurdles for automated solvers due to its vast array of potential configurations. Traditional exhaustive search techniques rapidly lose their effectiveness when applied to more complex, multi-move scrambles. This paper presents the design and implementation of a Rubik's Cube solver using classical and heuristic search algorithms. The cube is modeled as a state-space graph [6], where each configuration represents a node, and each valid move represents a transition between states.

In this study, three search algorithms, namely Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*), are implemented and comparatively analyzed. BFS guarantees optimal solutions but requires significant memory, while IDDFS reduces memory usage at the cost of repeated exploration. IDA* utilizes heuristic-based pruning to improve search efficiency.

The solver utilizes a compact state representation to efficiently manage memory. The performance of the algorithms is evaluated based on execution time, memory usage, and solution efficiency under controlled conditions [4]. The results indicate that BFS becomes impractical for deeper scrambles due to high memory requirements, whereas IDA* performs more efficiently by reducing unnecessary exploration through heuristic guidance.

Keywords - Rubik's Cube, BFS, IDDFS, IDA*, Heuristic search, Pattern database

1 INTRODUCTION

The Rubik's Cube is a widely studied combinatorial puzzle in computer science due to its large and complex state space [6]. A standard 3×3 Rubik's Cube has approximately 4.3×10^{19} possible configurations, making it a challenging problem for algorithmic solving approaches [5].

The problem can be represented as a state-space graph, where each configuration corresponds to a node and each move defines a transition [2]. The objective of solving the cube is to determine a sequence of moves that transforms a scrambled configuration into the solved state. However, the enormous number of possible states and high branching factor, make efficient state space exploration a significant challenge.

Traditional brute-force methods are impractical for solving the Rubik's Cube at higher depths because they use too much time and memory. Efficient search strategies are required to explore the state space effectively [2]. Algorithms like Breadth-First Search (BFS), Depth-First Search (DFS) and heuristic-based approaches have been explored for this problem.

This paper focuses on implementing and comparing different search algorithms, including Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*), to analyze their performance in solving the Rubik's Cube. The comparison is based on factors such as execution time, memory usage, and solution efficiency under controlled conditions.

2 RELATED WORK

The Rubik's Cube solving problem has been extensively studied in the domain of artificial intelligence and heuristic

search due to its large state space and computational complexity [2]. Early approaches primarily relied on uninformed search methods such as Breadth-First Search (BFS) and Depth-First Search (DFS). Breadth-First Search is noted for its ability to always identify the shortest solution path, though its high RAM consumption is a drawback. Conversely, Depth-First Search conserves memory but often struggles to locate optimal paths in a timely manner.

To overcome these limitations, Iterative Deepening Depth-First Search (IDDFS) was introduced as a hybrid approach that combines the advantages of BFS and DFS [3]. It performs repeated depth-limited searches, maintaining completeness while reducing memory usage.

A major advancement in solving the Rubik's Cube was introduced by Korf, who proposed the Iterative Deepening A* (IDA*) algorithm [1]. This approach uses admissible heuristics to guide the search process, significantly reducing the number of explored states compared to uninformed methods. Heuristic techniques allow the algorithm to focus on promising paths, improving overall efficiency.

Pattern databases are commonly employed to store precomputed partial solutions, which help in reducing the search effort by providing informed estimates [4]. These databases help reduce search time by providing heuristic estimates for reaching the goal state.

Modern approaches combine heuristic search algorithms with pruning techniques to efficiently handle large state-space problems [2]. These methods aim to strike a balance between time complexity, memory usage, and solution optimality.

This work builds upon these existing approaches by implementing and comparing BFS, IDDFS, and IDA* algorithms within a unified framework, focusing on their performance in terms of execution time, memory usage, and solution efficiency.

3 PROBLEM DEFINITION

The Rubik's Cube can be understood as a state space search problem [2], where different cube configurations represent different states. Given a scrambled configuration of the cube, the objective is to determine a sequence of valid moves that transforms it into the solved state. Each configuration of the cube represents a unique state, and each valid move corresponds to a deterministic transition between states [5].

A major challenge in solving the Rubik's Cube arises from its extremely large state space, which contains approximately 4.3×10^{19} possible configurations [5]. As the number of possible states grows exponentially with the depth of the search, exploring all configurations becomes computationally infeasible. Therefore, efficient search strategies are required to navigate the state space effectively.

Another important aspect of the problem is the trade-off between execution time and memory usage. Algorithms that guarantee optimal solutions often require significant memory, while memory-efficient approaches may increase computation time due to repeated exploration of states. Solution optimality is typically defined as the minimum number of moves required to reach the solved state [8] .

The problem addressed in this work is to design and analyze a system that accepts a scrambled Rubik's Cube as input and computes a valid sequence of moves to solve it efficiently, while balancing time complexity, memory usage, and solution quality under practical constraints.

4 METHODOLOGY

The Rubik's Cube is modeled as a state-space graph [2], where each possible arrangement of the cube is considered a node, and each valid move that changes the cube's state is represented as an edge connecting two nodes. The solver uses this graph to find a series of moves that change a scrambled cube into its solved state. To do this effectively, the system uses efficient methods for representing cube states and suitable search techniques.

4.1 Cube Representation

The cube is represented with a compact data structure that uses a nibble-based encoding method. This method reduces the memory needed to store each cube state by focusing on efficient encoding rather than keeping track of the full color details of each face.

This approach offers several benefits, including using less memory, making it faster to compare different cube states, and improving performance during search operations by using the computer's memory more efficiently. It also allows for quicker hashing and searching, which are important for algorithms that work with graphs.

4.2 Algorithms Implemented

Breadth-First Search (BFS): Breadth-First Search explores the state space in layers, ensuring that all configurations at a given depth is processed before moving to deeper levels [14]. However, it requires storing all the states at each level, which causes memory usage to grow quickly. This makes it unsuitable for solving more complex or deeply scrambled cubes.

Iterative Deepening Depth-First Search (IDDFS): By executing a series of progressively deeper searches, IDDFS successfully blends the memory conservation of DFS with the completeness of BFS [2]. This method uses less memory than BFS and still guarantees finding a solution if one exists. However, it may explore the same states multiple times, which can slow down the process.

Iterative Deepening A* (IDA*): IDA* is a heuristic-driven search technique that guides exploration using cost estimation [1], enabling more efficient traversal of the state space.
It calculates the cost of reaching a state using a formula that combines two parts:

$$f(n) = g(n) + h(n)$$

where g(n) is the actual cost to reach the current state and h(n) is an estimate of the cost to get from the current state to the solved state [1].

IDA* uses this information along with a depth limit to guide the search more effectively, reducing the number of states that need to be explored compared to other methods.

4.3 Heuristic Optimization

To enhance the efficiency of the search process, a heuristic approach using a pattern database is applied [4].

Specifically, a corner pattern database is used, which stores precomputed information about the minimum number of moves needed to solve different corner configurations of the cube. This database is built using Breadth-First Search and provides reliable estimates of the number of moves needed to reach the solution [6].

By using these estimates, the search algorithm can avoid unnecessary exploration of states, making the overall search process more efficient, especially for highly scrambled cubes.

5 IMPLEMENTATION

The proposed Rubik's Cube solver is implemented in Java using object-oriented design principles.

The system is organized in a modular way, which separates the representation of the cube, the search algorithms, and the heuristic components. This design enhances the system's maintainability and allows for easy comparison of various solving methods.

5.1 System Architecture

The system comprises several components, each handling a specific function:

1) Rubik's Cube: This component represents the basic model of the cube and offers functions like applying moves and checking whether the cube is fully solved.

2) Bitboard Representation: This part uses a compact method to encode the cube's state, which helps in using less memory and comparing states more efficiently.

3) Solver Modules: These include various search algorithms such as BFS, IDDFS, and IDA*, each implemented as a separate class.

4) Pattern Database: This provides support for heuristics by storing precomputed data that is used in the IDA* algorithm.

This modular setup allows for the independent development and testing of different algorithms.

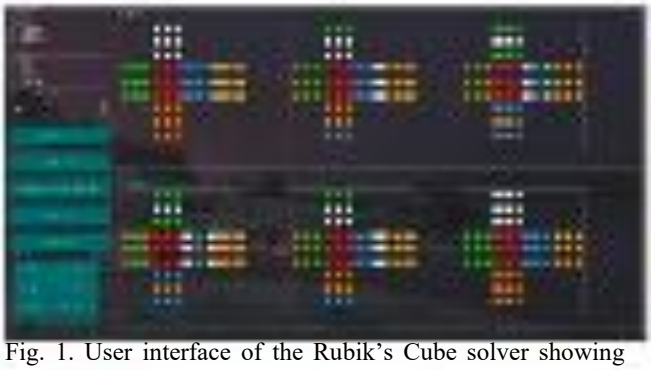


Fig. 1. User interface of the Rubik's Cube solver showing cube visualization, shuffle operations, and solution steps.

5.2 Algorithm Implementation

Each search algorithm is implemented as a distinct solver class. The solvers interact with the cube's representation by applying valid moves and examining the resulting states.

For instance, a depth-first search method explores possible move sequences recursively up to a predetermined depth. The solver keeps track of the moves and uses backtracking to investigate other paths.

5.3 Pseudocodes

1) Pseudocode for BFS

```
BFS
function BFS (start):
    create an empty queue
      create an empty set for visited
states


    enqueue start into queue
    mark start as visited

    while queue is not empty:
            currentState = dequeue from
queue

        if currentState is cost:
            return solution

                for each nextState of
currentState:
            if nextState is not visited:
                        mark nextState as
visited
                  enqueue nextState into
queue

    return NOT_FOUND
```

2) Pseudocode for IDDFS

```
IDDFS
function IDDFS (start):
    for depth = 0 to max_depth:
        result = DLS (start, depth)
        if result == FOUND:
            return solution

    return NOT_FOUND


function DLS (currentState, depth):
     if depth == 0 and currentState is
cost:
        return FOUND

    if depth > 0:
                for each nextState of
currentState:
               result = DLS (nextState,
depth - 1)
            if result == FOUND:
                return FOUND

    return NOT_FOUND
```

3) Pseudocode for IDA*

```
    function IDA*(start):
    limit = heuristic(start)
    while True:
            t = exploreState (start, 0,
limit)

        if t == FOUND:
            return solution
        if t == infinity:
            return NOT_FOUND
        limit = t

function   exploreState   (currentState,
cost, limit):
    f = cost + heuristic(currentState)
    if f > limit:
        return f
    if currentState equals goalState:
        return FOUND
    min = infinity
      for each nextState generated from
currentState
           t = exploreState (nextState,
cost+1, limit)
        if t == FOUND:
            return FOUND
        if t < min:
            min = t
    return min
```

5.4 Complexity Analysis

The efficiency of the implemented algorithms can be evaluated in terms of their time and space requirements [2]. Let b represent the branching factor (approximately 18 for the Rubik's Cube), and d represent the depth of the solution.

1) Breadth-First Search (BFS):
Time Complexity: $O(b^d)$
Space Complexity: $O(b^d)$

2) Iterative Deepening Depth-First Search (IDDFS):
Time Complexity: $O(b^d)$
Space Complexity: O(b*d)

3) Iterative Deepening A* (IDA*):
Time Complexity: $O(b^d)$
Space Complexity: O(d)

These complexities indicate that BFS becomes impractical for large depths due to high memory usage, while IDA* provides better scalability with reduced memory requirements.

6 EXPERIMENTAL SETUP

The performance of the implemented algorithms was evaluated using a Java-based implementation executed on a standard computing environment. The system was tested on a machine equipped with an Intel i5 processor and 8 GB of RAM.

To analyze the behavior of the algorithms under different conditions, multiple test cases were generated using randomly scrambled cube configurations. The scramble depth was varied within a controlled range (approximately 3 to 10 moves) to observe the performance of each algorithm at different levels of complexity. Each test case was executed multiple times to obtain consistent observations.

The evaluation of the algorithms was based on the following performance metrics:

1) Execution Time: Time required to compute the solution.

2) Memory Usage: The Amount of memory consumed during the search process.

3) Solution Length: Number of moves required to solve the cube.

This setup allows a comparative analysis of different algorithms in terms of efficiency and scalability.

7 RESULTS

The performance of the implemented algorithms was evaluated based on execution time, memory usage, and solution efficiency across different scramble depths. The results demonstrate the comparative behavior of BFS, IDDFS, and IDA* under increasing problem complexity.

7.1 Comparative Performance

This represents a qualitative comparison of the algorithms based on observed performance characteristics.

Table I. Comparison of search algorithms based on performance metrics

Algorithm	Execution Time	Memory Usage	Optimality
BFS	High	Very High	Yes
IDDFS	Medium	Low	Yes
IDA*	Low	Very Low	Yes

7.2 Time v/s Depth Analysis

The execution time of each algorithm was evaluated for increasing scramble depths. The observed values indicate that the performance of BFS degrades rapidly due to exponential growth in memory requirements. IDDFS performs better in terms of memory but exhibits increased computation time due to repeated exploration of states. IDA* shows more stable performance due to heuristic-guided pruning.

Table II. Execution time of algorithms for different scramble depths

Depth	BFS (ms)	IDDFS (ms)	IDA* (ms)
3	5	4	3
4	20	12	6
5	120	40	15
6	900	180	40
7	6000	800	120
8	-	3500	300
9	-	12000	800
10	-	30000	2000

7.3 Graph Representation

The graph below shows a performance comparison:

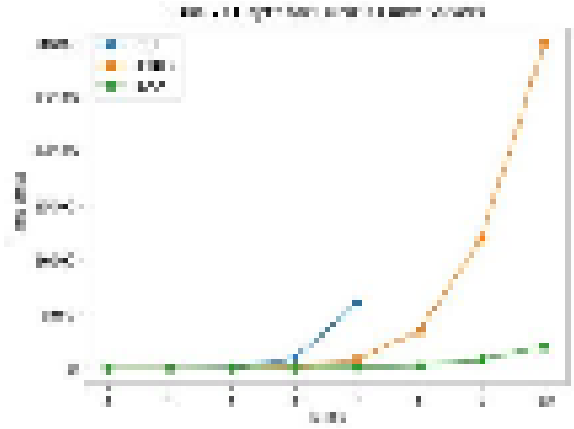


Fig. 2. Time vs Depth for Rubik's Cube Solvers

7.4 Observations

The results highlight several important trends:

1) BFS becomes impractical beyond moderate depths due to exponential memory requirements.

2) IDDFS reduces memory consumption but increases computation time due to repeated searches.

3) IDA* demonstrates better scalability and efficiency by using heuristic-guided pruning.

Overall, heuristic-based approaches such as IDA* provide a more balanced trade-off between time and memory usage for solving large state-space problems like the Rubik's Cube.

8 CONCLUSIONS

This paper presents a comparative study of different search algorithms for solving the Rubik's Cube, focusing on Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*). The problem was modeled as a state-space search, and the algorithms were evaluated based on execution time, memory usage, and solution efficiency under controlled experimental conditions.

The results indicate that BFS, although capable of producing optimal solutions, becomes impractical for larger scramble depths due to its high memory requirements. IDDFS improves memory efficiency by using depth-limited search; however, it introduces additional computational overhead due to repeated exploration of states. In contrast, IDA* demonstrates better overall performance by incorporating heuristic-based pruning, which reduces unnecessary exploration and allows the algorithm to scale more effectively with increasing depth.

The use of compact state representation further contributes to improved efficiency by reducing memory usage and enabling faster state operations. Overall, the study highlights that heuristic-based approaches are more suitable for solving large combinatorial problems such as the Rubik's Cube, as they provide a balanced trade-off between time complexity and memory consumption.

Future work may focus on enhancing the solver through improved heuristic techniques, optimization strategies, or parallel implementations to further improve performance and scalability.

9 REFERENCES

[1] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," Artificial Intelligence, vol. 27, no. 1, pp. 97–109, 1985.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[3] D. Singmaster, Notes on Rubik's Cube. Enslow Publishers, 1981.

[4] J. C. Culberson and J. Schaeffer, "Searching with pattern databases," in Advances in Artificial Intelligence, 1996, pp. 402–416.

[5] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "The diameter of the Rubik's Cube group is twenty," SIAM Journal on Discrete Mathematics, vol. 27, no. 2, pp. 1082–1105, 2013.

[6] R. E. Korf, "Finding optimal solutions to Rubik's Cube using pattern databases," AAAI, 1997.

[7] H. Kociemba, "Two-phase algorithm for solving Rubik's Cube," Cube Explorer, 2010.

[8] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "The diameter of the Rubik's Cube group is twenty," SIAM Journal, 2013.

[9] S. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach," 3rd ed., Pearson, 2010.

[10] J. Pearl, "Heuristics: Intelligent search strategies," Addison-Wesley, 1984.

[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1959.

[12] N. J. Nilsson, "Principles of Artificial Intelligence," Morgan Kaufmann, 1980.

[13] J. Culberson and J. Schaeffer, "Pattern databases," Computational Intelligence, 1998.

[14] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for heuristic determination," IEEE Transactions, 1968.

[15] M. Sipser, "Introduction to the theory of computation," Cengage Learning, 2012.

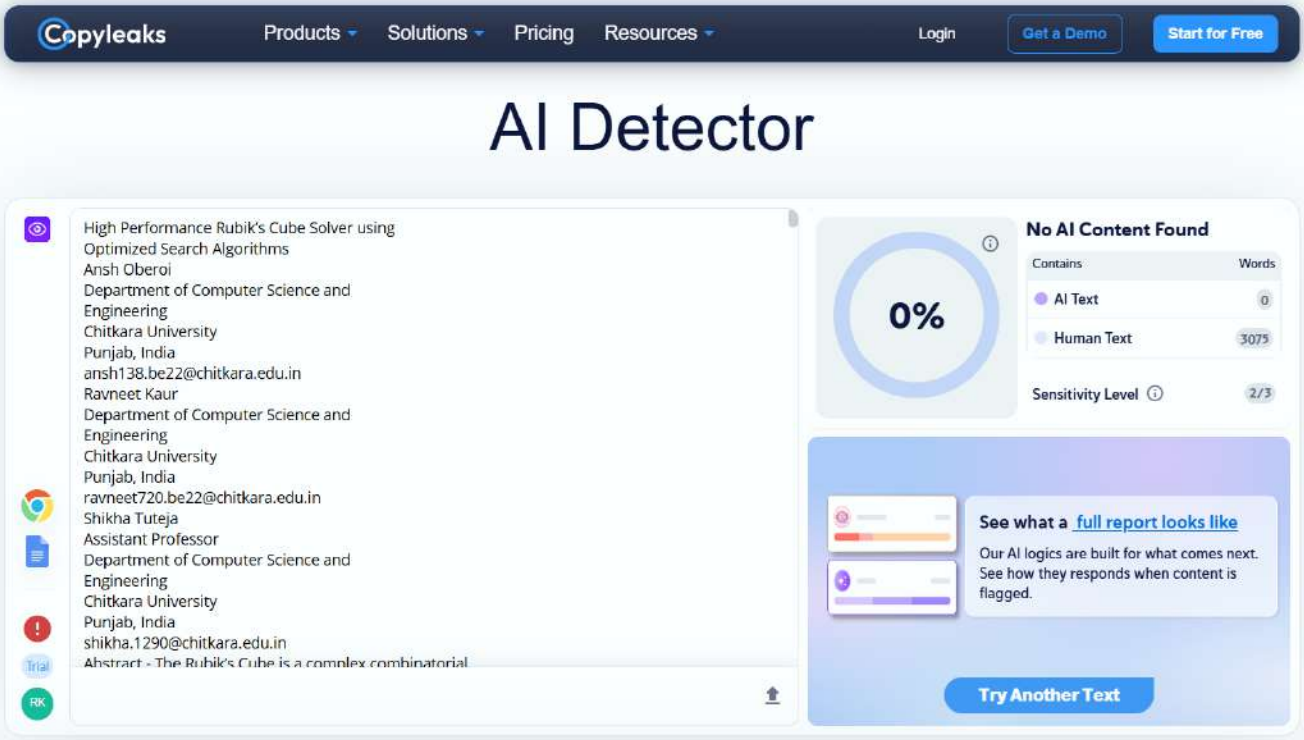
[16] D. E. Knuth, "The art of computer programming," Addison-Wesley, 1997.

[17] A. Newell and H. A. Simon, "Human problem solving," Prentice Hall, 1972.

[18] G. Brassard and P. Bratley, "Fundamentals of algorithmics," Prentice Hall, 1996.

[19] T. H. Cormen et al., "Introduction to algorithms," MIT Press, 2009.

[20] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning," MIT Press, 2016.


Copyleaks
Products
Solutions
Pricing
Resources
Login
Get a Demo
Start for Free
AI Detector
High Performance Rubik's Cube Solver using
Optimized Search Algorithms
Ansh Oberoi
Department of Computer Science and
Engineering
Chitkara University
Punjab, India
ansh138.be22@chitkara.edu.in
Ravneet Kaur
Department of Computer Science and
Engineering
Chitkara University
Punjab, India
ravneet720.be22@chitkara.edu.in
Shikha Tuteja
Assistant Professor
Department of Computer Science and
Engineering
Chitkara University
Punjab, India
shikha.1290@chitkara.edu.in
Abstract - The Rubik's Cube is a complex combinatorial
Trial
RK
0%
No AI Content Found
Contains
Words
AI Text
0
Human Text
3075
Sensitivity Level
2/3
See what a full report looks like
Our AI logics are built for what comes next.
See how they responds when content is
flagged.
Try Another Text

ResearchPaper_RavneetKaur.docx

ATLAS SkillTech University

Document Details

Submission ID
trn:oid:::3618:136196234

Submission Date
Apr 22, 2026, 11:48 PM GMT+5:30

Download Date
Apr 22, 2026, 11:55 PM GMT+5:30

File Name
ResearchPaper_RavneetKaur.docx

File Size
185.6 KB

6 Pages

3,071 Words

16,961 Characters

25% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

7 AI-generated only 25%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

High Performance Rubik's Cube Solver using Optimized Search Algorithms

Ansh Oberoi
Department of Computer Science and Engineering
Chitkara University
Punjab, India
ansh138.be22@chitkara.edu.in

Ravneet Kaur
Department of Computer Science and Engineering
Chitkara University
Punjab, India
ravneet720.be22@chitkara.edu.in

Shikha Tuteja
Department of Computer Science and Engineering
Chitkara University
Punjab, India
shikha.1290@chitkara.edu.in

Abstract **-** The Rubik's Cube is a complex combinatorial puzzle that presents many difficulties for automated solvers due to a large array of potential configurations. As the number of moves in scrambles increases, traditional exhaustive techniques are losing effectiveness on more complex problems due to the exponential growth of the search space.

In this paper, we present the design and implementation of a Rubik's Cube solver using heuristic search algorithms. The main purpose is to implement a high-performance Rubik's solver in java which ensure best solution by minimizing the complexity of the cube. The cube is represented as a state-space graph [6], where a node is used for each configuration, and transitions between the states are shown by each valid move.

This study compares three search algorithms: Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*). BFS finds optimal solutions but uses much memory. IDDFS uses less memory, but requires many repeated searches. IDA* is best and most efficient due to heuristics.

Cube is presented as a compact state to prevent the usage of memory. The performance of the algorithm is measured by time, memory used, and solution quality under some of the applied conditions [4]. BFS has done the deep searches, which consumed much memory. IDA* uses heuristics to avoid unnecessary searches, making it more efficient.

Keywords **-** Rubik's Cube, BFS, IDDFS, IDA*, Heuristic search, Pattern database

1 INTRODUCTION

In computer science, a combinatorial puzzle is studied known as Rubik's Cube. It is a very large and compact state space [6]. It has approximately 4.3×10^19 possible configurations in a standard 3×3 cube, which makes it a challenging problem for algorithmic solving approaches [5].

In this, each configuration corresponds to a node, and a transition is defined by each move, as this problem is stated as a state-space graph [2]. The main aim is to solve the scrambled cube by determining the number of moves. However, the enormous number of possible states and high branching factor make efficient state space exploration a significant challenge.

The older brute-force methods are not efficient as they use too much memory and time for solving the cube at higher depths. To overcome this, some more efficient strategies are required to explore the state space [2]. Algorithms like Breadth-First Search (BFS), Depth-First Search (DFS) and heuristic-based approaches have been explored for this problem.

This paper focuses on implementing and comparing different search algorithms, including Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*), to analyze their performance in solving the Rubik's Cube. The comparison is based on factors such as execution time, memory usage, and solution efficiency under controlled conditions.

2 RELATED WORK

The Rubik's Cube problem has been studied in the area of artificial intelligence and heuristic search due to its large state space and algorithmic complexity [2]. The two search methods, Breadth-First Search (BFS) and Depth-First Search (DFS), were the approaches primarily used. Breath-first search is known for its competence to identify the shortest path towards the solution, even though it has the drawback of high RAM consumption. On the other hand, Depth-first search struggles to locate the best paths on time, but it was beneficial in conserving a lot of memory.

To conquer these limitations, Iterative Deepening Depth-First Search (IDDFS) was introduced as a hybrid approach that combines the advantages of both BFS and DFS [3]. It is helpful as it maintains completeness by reducing the memory usage. It executes constant-depth-limited searches.

Korf was the one who invented the Iterative Deepening A* (IDA*) algorithm, a new advancement in solving the Rubik's Cube [1]. This approach uses acceptable heuristics to guide the search process, also reducing the number of explored states compared to uninformed methods. Now, the algorithm focuses on promising paths and improves the overall efficiency by using these heuristic techniques.

The pattern databases are commonly engaged to store precomputed partial solutions, which also helps in providing informed estimates by reducing the search effort [4]. These databases help reduce the search time by providing heuristic estimates for reaching the goal state.

These large state space problems have been efficiently handled by the modern approaches, which have combined the heuristic search algorithms with pruning techniques [2]. These methods are very useful in making a balance between time complexity, memory usage and solution optimality.

In this work, provided on the already existing approaches by implementing and comparing BFS, IDDFS and IDA* algorithms within a common framework, focusing on their performance in terms of execution time, memory usage, and solution efficiency.

3 PROBLEM DEFINITION

The Rubik's Cube can be understood as a state space search problem [2], where different cube configurations represent different states. We are given a scrambled configuration of the cube; the aim is to determine a sequence of valid moves so that we can get a solved state of the cube. Each valid move corresponds to a deterministic transition between states, and each configuration of the cube represents a unique state [5].

A major problem in solving the Rubik's Cube is due to its large state space, which contains approximately. 4.3×10^{19} possible configurations [5]. To explore all the configurations is computationally not practical as the depth of the search grows rapidly with the number of possible states. There is a need to implement those search strategies that are efficient and can easily navigate the state space.

The balance between execution time and memory usage is another problem. Algorithms with memory-efficient approaches increase the computation time during repeated exploration of states, resulting in optimal solutions, but much memory is used. To reach the solved state, we need to perform solution optimality, in which the minimum number of moves is required [8].

The problem considered in this work is to be solved while balancing time complexity, memory usage and solution quality with certain constraints, a scrambled Rubik's Cube accepted as an input, which needs to be designed to find the minimum number of moves to solve it efficiently.

4 METHODOLOGY

The Rubik's Cube is modeled as a state-space graph [2], where an edge connecting two nodes is a valid move that changes the cube's state, and a node is each possible arrangement of the cube. The solver uses this graph to find a series of moves that change a scrambled cube into its answered state. To get this all done effectively, efficient methods are used for suitable search techniques and for representing cube states.

4.1 Cube Representation

The Nibble-based encoding method was used to represent a compact data structure. The full color details of each face were no longer required because this method is focused on efficient encoding, which helped in reducing the memory needed to store each cube state.

There are many benefits of using this approach, including less memory usage, comparison of the different cube states is

much faster, and the performance of the search operations has improved by using the computer's memory more efficiently. As these algorithms work more with the graphs, allowed much quicker hashing and searching.

4.2 Algorithms Implemented

1) Breadth-First Search (BFS): It explores the state space in layers, which means that all configurations at a given depth are analyzed before moving to deeper levels [14]. In this, the memory usage grows quickly, as it stores all the states at each level. So, the more complex or deeply scrambled cubes are not suitable for solving.

2) Iterative Deepening Depth-First Search (IDDFS): By executing a series of progressively deeper searches, IDDFS successfully blends the memory conservation of DFS with the completeness of BFS [2]. This method uses less memory than BFS and still guarantees finding a solution if one exists. However, it may explore the same states multiple times, which can slow down the process.

3) Iterative Deepening A* (IDA*): This uses cost evaluation, making it more efficient and also good for exploration. It is a search technique that uses a heuristic-guided method.

It calculates the cost of reaching a state using a formula that combines two parts:

$$\mathbf{f(n) = g(n) + h(n)}$$

where g(n) is the actual cost to reach the current state and h(n) is an estimate of the cost to get from the current state to the solved state [1].

IDA* uses this information along with a depth limit to guide the search more effectively, reducing the number of states that need to be explored compared to other methods.

4.3 Heuristic Optimization

The heuristic approach of the pattern database is best to enhance the efficiency of the search process [4]. To solve different corner configurations of the cube with a minimum number of moves, a corner pattern database is used, which stores precomputed information. Breadth-first search is used in building this database, which provides the minimum number of moves needed to reach the solution.

For highly scrambled cubes, the search process can get more efficient, and unnecessary exploration of states can also be avoided in the search algorithm just by using these estimates.

5 IMPLEMENTATION

The proposed Rubik's Cube solver is implemented in Java using object-oriented design principles.

The system is organized in a modular way, which separates the representation of the cube, the search algorithms, and the heuristic components. The design has already enhanced the maintainability and made it easy to solve various issues.

5.1 System Architecture

Some of the several forms of systems with functions are as follows:

1) Rubik's Cube: This is the basic model of the cube. It has some functions, such as applying minimum moves and determining whether the cube is solved properly.

2) Bitboard Representation: This is the compact method of solving the cube's state. It uses the least memory through the process and compares states more easily.

3) Solver Modules: These include various search algorithms such as BFS, IDDFS, and IDA*, each implemented as a separate class.

4) Pattern Database: This provides support for heuristics by storing precomputed data that is used in the IDA* algorithm.

The individual development and testing of these algorithms is allowed through this setup.

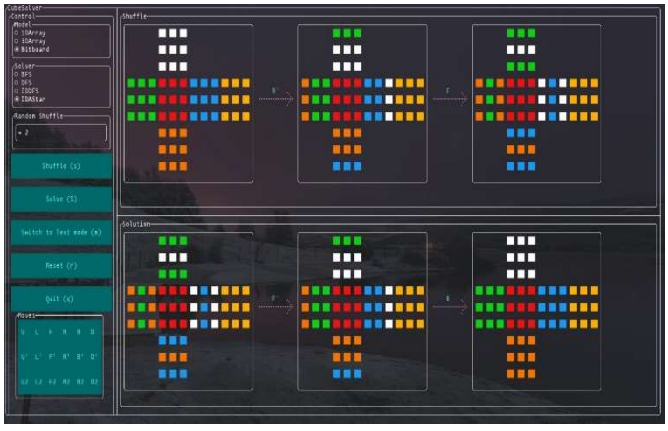


Fig. 1. User interface of the Rubik's Cube solver showing cube visualization, shuffle operations, and solution steps.

5.2 Algorithm Implementation

Each of the search algorithms is implemented as a distinct solver class. This solver interacts directly with the cube's representation by applying valid moves and also examining the resulting states.

For instance, a depth-first search method explores possible move sequences recursively up to a predetermined depth. The solver keeps track of the moves and uses backtracking to investigate other paths.

5.3 Pseudocodes

1) Pseudocode for BFS

```
BFS
function BFS (start):
    Create an empty queue
    Create an empty set for visited states

    enqueue start into the queue
    mark start as visited

    while queue is not empty:
        currentState = dequeue from queue

        if currentState is cost:
            return solution

        for each nextState of
currentState:
            if nextState is not visited:
                mark nextState as visited
                enqueue nextState into
the queue

    return NOT_FOUND
```

2) Pseudocode for IDDFS

```
IDDFS
function IDDFS (start):
    for depth = 0 to max_depth:
        result = DLS (start, depth)
        if result == FOUND:
            return solution

    return NOT_FOUND


function DLS (currentState, depth):
    if depth == 0 and currentState is
cost:
        return FOUND

    if depth > 0:
        for each nextState of
currentState:
            result = DLS (nextState,
depth - 1)
            if result == FOUND:
                return FOUND

    return NOT_FOUND
```

3) Pseudocode for IDA*

```
        function IDA*(start):
    limit = heuristic(start)
    while True:
        t = exploreState (start, 0,
limit)
        if t == FOUND:
            return solution
        if t == infinity:
            return NOT_FOUND
        limit = t

function exploreState (currentState,
cost, limit):
    f = cost + heuristic(currentState)
    if f > limit:
        return f
    if currentState equals goalState:
        return FOUND
    min = infinity
    for each nextState generated from
currentState
        t = exploreState (nextState,
cost+1, limit)
        if t == FOUND:
            return FOUND
        if t < min:
            min = t
    return min
```

5.4 Complexity Analysis

The efficiency of the following algorithms is demonstrated below in terms of their time and space complexity [2].
Let b represent the branching factor (approximately 18 for the Rubik's Cube), and d represent the depth of the solution.

1) Breadth-First Search
Time Complexity: $O(b^d)$
Space Complexity: $O(b^d)$

2) Iterative Deepening Depth-First Search
Time Complexity: $O(b^d)$
Space Complexity: O(b*d)

3) Iterative Deepening A*
Time Complexity: $O(b^d)$
Space Complexity: O(d)

Therefore, these complexities tell us that BFS becomes impractical for large depths due to high memory usage. On the other hand, IDA* provides much better scalability with reduced requirements for memory.

6 EXPERIMENTAL SETUP

The performance of the implemented algorithms was evaluated using a Java-based implementation executed on a standard computing environment. The system was tested on a machine equipped with an Intel i5 processor and 8 GB of RAM.

To dissect the behavior of the algorithms under different conditions, multiple test cases were generated using aimlessly climbed cell configurations. The scramble depth was varied within a controlled range (roughly 3 to 10 moves) to observe the performance of each algorithm in different situations of complexity. To obtain stable observations, each test case was executed multiple times.

The following metrics were grounded to evaluate the algorithms:

1) Execution Time: Time required to compute the solution.

2) Memory Usage: The Amount of memory consumed during the search process.

3) Solution Length: Number of moves required to solve the cube.

This setup allows a relative analysis of different algorithms in terms of effectiveness and scalability.

7 RESULTS

Based on execution time, memory usage and solution efficiency, the performance of the implemented algorithms was evaluated across different scrambled paths. The results demonstrate the comparative behavior of BFS, IDDFS, and IDA* as problems become more complex.

7.1 Comparative Performance

This represents a qualitative comparison of the algorithms based on observed performance characteristics.

Table I. Comparison of search algorithms based on performance metrics

Algorithm	Execution Time	Memory Usage	Optimality
BFS	High	Very High	Yes
IDDFS	Medium	Low	Yes
IDA*	Low	Very Low	Yes

7.2 Time v/s Depth Analysis

For each increasing scramble depth, all the algorithms' execution time is evaluated. The observed values show us that for BFS, it is increasing promptly due to expanding growth in memory usage. IDDFS, due to repeated state exploration, has increased the computation, but performs better in terms of memory. IDA* shows the best results with consistent values due to heuristic-guided pruning.

Table II. Execution time of algorithms for different scramble depths

Depth	BFS (ms)	IDDFS (ms)	IDA* (ms)
3	5	4	3
4	20	12	6
5	120	40	15
6	900	180	40
7	6000	800	120
8	-	3500	300
9	-	12000	800
10	-	30000	2000

7.3 Graph Representation

The graph below shows a performance comparison between time and depth:

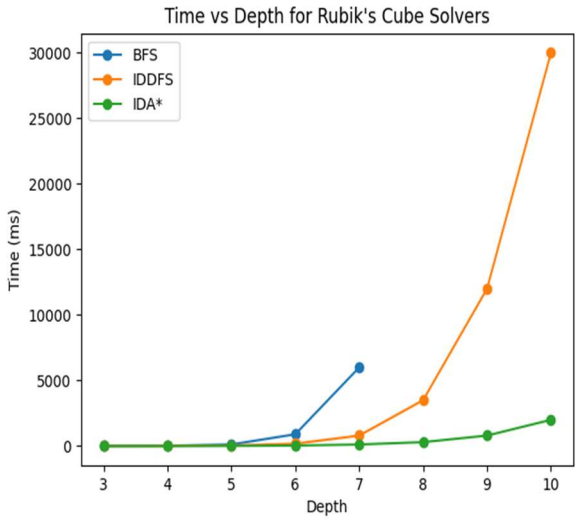


Fig. 2. Time vs Depth for Rubik's Cube Solvers

7.4 Observations

The observation concluded from the result is as follows:

1) BFS, due to its exponential memory needs it becomes impractical for the higher depths.

2) IDDFS, due to its continuously repeated searches, increases the time but helps reduce memory.

3) IDA* using the heuristic-guided pruning gives better scalability and increased efficiency.

Overall, heuristic-based approaches such as IDA* provide a more balanced trade-off between time and memory usage for solving large state-space problems like the Rubik's Cube.

8 CONCLUSIONS

This paper is focused mainly on the three algorithms, such as Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*). In this comparative study, the solved state of the Rubik's cube. This is modeled as a state space search problem, and the evaluation of the algorithms was based on execution time, solution efficiency and memory usage under controlled supervised conditions.

The results depict that BFS, due to its high memory consumption, is impractical for larger scramble depths, although capable of producing the best solutions. IDDFS, due to repeated exploration of states, introduces additional computational overhead; however, it improves memory efficiency by using depth-limited search. In comparison, IDA* with adding depth allows the algorithm to gauge more effectively, demonstrating better overall performance by incorporating heuristic-grounded pruning.

The reduced memory operations and rapid state operations happened due to the use of compact state representation, which also contributed to increasing effectiveness. Overall, the study highlights that Rubik's Cube provides a balanced relationship between time complexity and memory consumption, so to solve these large combinatorial problems, heuristic-grounded approaches are best suitable.

Future work may concentrate on enhancing the solver through better heuristic ways, optimisation strategies, or corresponding executions to further improve performance and scalability.

9 REFERENCES

[1] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," Artificial Intelligence, vol. 27, no. 1, pp. 97–109, 1985.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[3] D. Singmaster, Notes on Rubik's Cube. Enslow Publishers, 1981.

[4] J. C. Culberson and J. Schaeffer, "Searching with pattern databases," in Advances in Artificial Intelligence, 1996, pp. 402–416.

[5] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "The diameter of the Rubik's Cube group is twenty," SIAM Journal on Discrete Mathematics, vol. 27, no. 2, pp. 1082–1105, 2013.

[6] R. E. Korf, "Finding optimal solutions to Rubik's Cube using pattern databases," AAAI, 1997.

[7] H. Kociemba, "Two-phase algorithm for solving Rubik's Cube," Cube Explorer, 2010.

[8] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "The diameter of the Rubik's Cube group is twenty," SIAM Journal, 2013.

[9] S. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach," 3rd ed., Pearson, 2010.

[10] J. Pearl, "Heuristics: Intelligent search strategies," Addison-Wesley, 1984.

[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1959.

[12] N. J. Nilsson, "Principles of Artificial Intelligence," Morgan Kaufmann, 1980.

[13] J. Culberson and J. Schaeffer, "Pattern databases," Computational Intelligence, 1998.

[14] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for heuristic determination," IEEE Transactions, 1968.

[15] M. Sipser, "Introduction to the theory of computation," Cengage Learning, 2012.

[16] D. E. Knuth, "The art of computer programming," Addison-Wesley, 1997.

[17] A. Newell and H. A. Simon, "Human problem solving," Prentice Hall, 1972.

[18] G. Brassard and P. Bratley, "Fundamentals of algorithmics," Prentice Hall, 1996.

[19] T. H. Cormen et al., "Introduction to algorithms," MIT Press, 2009.

[20] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning," MIT Press, 2016.